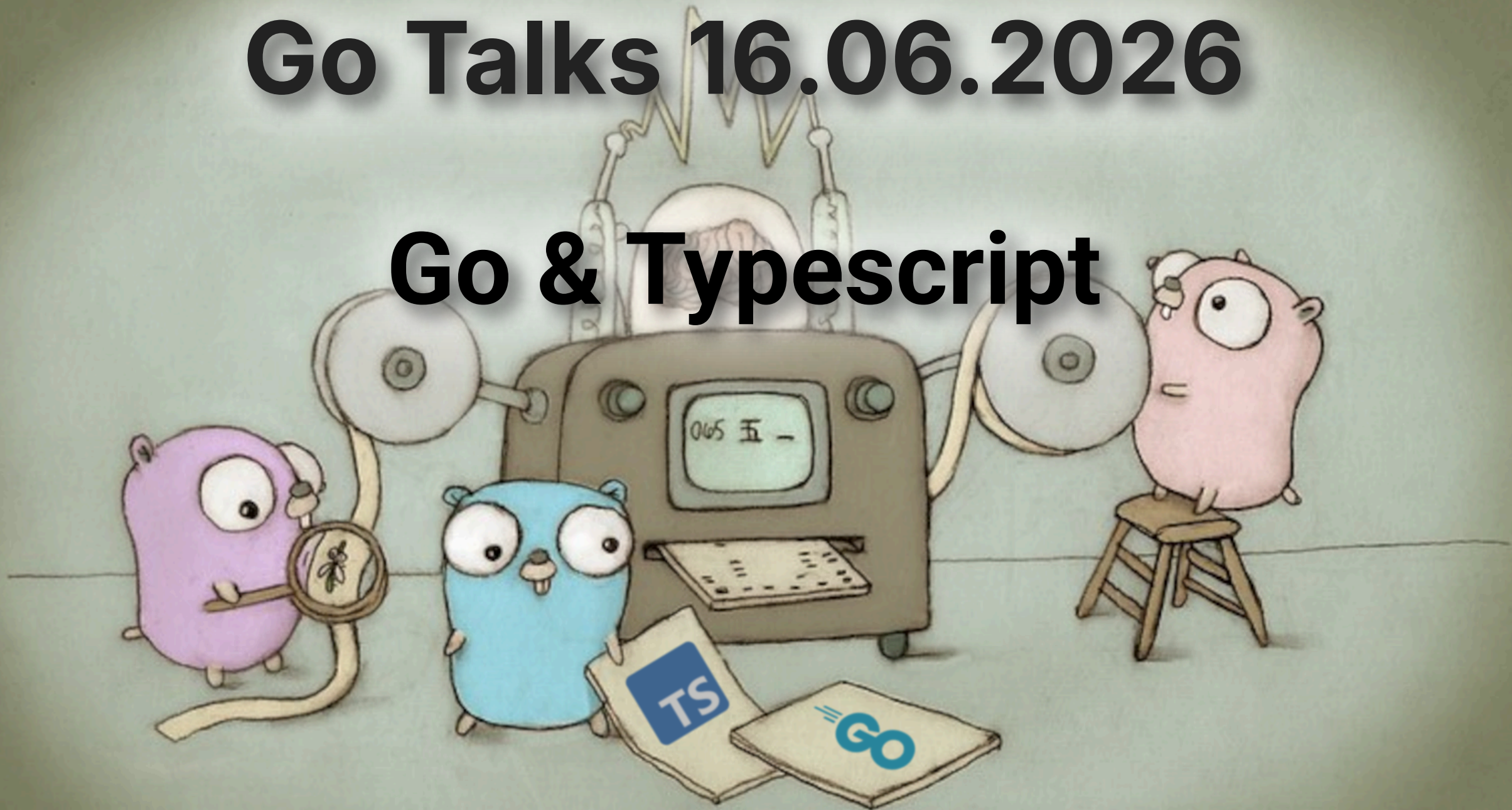


Go Talks 16.06.2026

Go & Typescript





How to reach us



meetup.com/Golang-ZG/



[@golangzg](https://www.youtube.com/@golangzg)



github.com/golang-zg/meetups

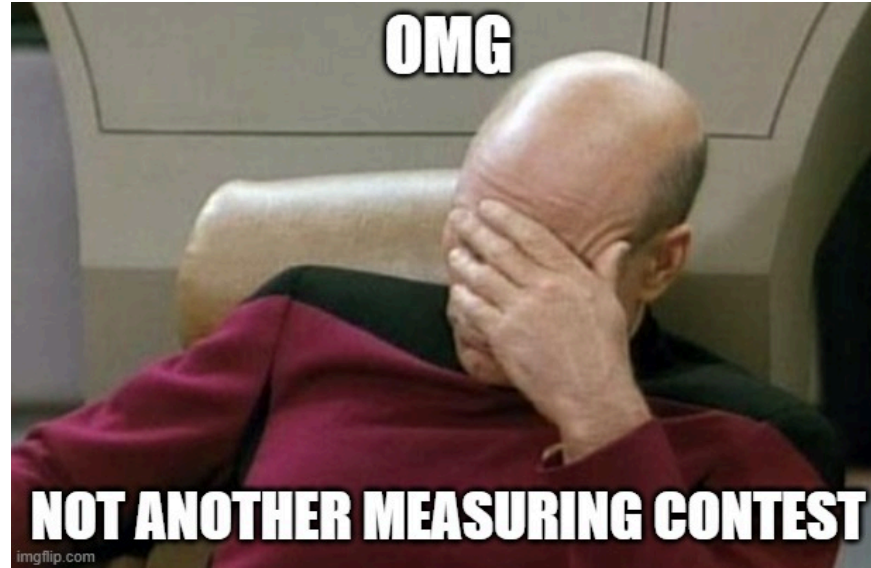


[@golangzg.bsky.social](https://bsky.app/@golangzg.bsky.social)

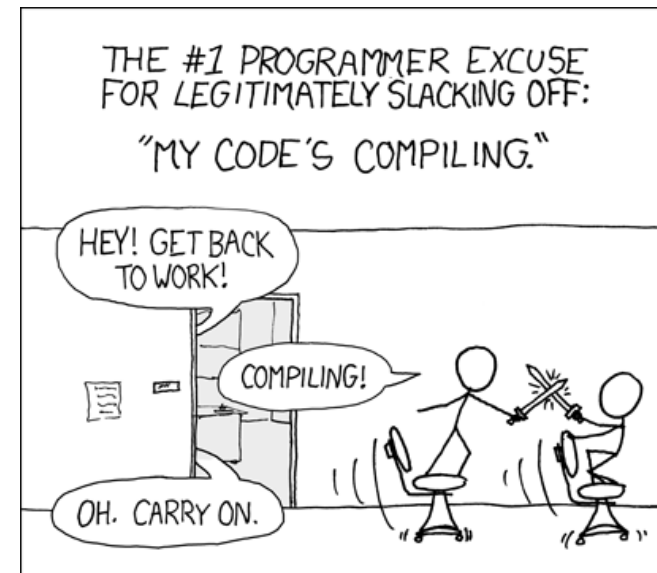


[invite.slack.golangbridge.org #croatia](https://invite.slack.golangbridge.org/#croatia)

Go & Typescript



- **Created at Google (2007)**
 - by Robert Griesemer, Rob Pike & Ken Thompson;
- **v1.0** in 2012
- **Compiled, statically typed**
 - builds to a single self-contained binary
- **Built-in concurrency:**
 - goroutines and channels
 - CSP-style - Communicating Sequential Processes
- **Garbage collected** with a low-latency collector
- **Fast compilation** and a strong, batteries-included standard library
- **Simple by design** - small spec, `gofmt` for one canonical style
- **Great tooling:** `go build`, `go test`, modules, race detector
- Popular for **cloud, CLIs, and network services**



<https://xkcd.com/303/>

Runs almost everywhere:

- Linux, macOS, Windows, FreeBSD,
- `amd64`, `arm64`, `386`, `arm`, `riscv64`, `wasm` ...
- `GOOS=none` → soon™

Cross-compile from any host - `go tool dist list`

- `GOOS=linux GOARCH=arm64 go build`
- `GOOS=windows GOARCH=amd64 go build`

Single self-contained binary

`CGO_ENABLED=0` keeps builds fully static and portable across distros

WebAssembly:

- `GOOS=js GOARCH=wasm` (browser)
- `GOOS=wasip1 GOARCH=wasm` (WASI)

One command, batteries included

- `go build` / `go run` / `go install` / `go test`
- `go test`, `go fmt`, `go vet`, `go generate`
- `go mod + go.sum`
 - modules give reproducible dependency management
- `go doc` & `pkg.go.dev`
 - documentation generated straight from source
- `go tool` (`pprof`, `trace`, `dist`)
 - profiling, execution tracing and the platform list

- **goroutines**
 - lightweight threads; spin up thousands with `go f()`
- **channels**
 - typed, safe message passing
 - *"Don't communicate by sharing memory; share memory by communicating"*
- **-race detector**
 - catch data races automatically in tests

- **net/http**
 - production-grade web server & client, no framework required
- **batteries included**
 - `encoding/json`, `crypto`, `io`, `time`, `os`, `regexp` ...
- **testing**
 - tests, benchmarks and fuzzing in the box
- **backwards compatible**
 - the Go 1 compatibility promise

- **compiled to native code**
 - no interpreter, no JIT warm-up
- **low, predictable memory**
 - compact structs, value types, no VM overhead
- **efficient garbage collector**
 - concurrent, sub-millisecond pauses
- **great for CPU-bound & concurrent work**
 - scales across cores
- **fast builds**
 - whole-program compilation in seconds

- **static typing**
 - errors caught at compile time, not in production
- **structural interfaces**
 - satisfied implicitly; no `implements` keyword
- **zero values**
 - every type has a sane default; no `undefined`
- **composition over inheritance**
 - embed types, no class hierarchies
- **explicit error values**
 - errors are values, not exceptions

- **Cloud-native backbone**
 - Docker, Kubernetes, Terraform, Prometheus
- **Networking & infrastructure**
 - proxies, CLIs, control planes, APIs
- **Used by**
 - Google, Cloudflare, Uber, HashiCorp, Dropbox ...
- **CLIs & dev tools**
 - fast startup + single binary make distribution trivial
- **Microservices**
 - small footprint, high concurrency

- May 1995. by Brendan Eich, initially called Mocha
- pressure from Netscape management
 - built in 10 days?
 - bring interactivity to the Netscape Navigator 2.0
 - beginner-friendly "scripting companion" to Java
- early language quirks which could never be removed from language
- however, Javascript spread to all parts of the software stack



```
> true == 1
true

> true === 1
false

> typeof null
'object'
```

```
> [] == ![]
true

> [] + []
''

> [1, 2, 3] + [1]
'1,2,31'
```



TypeScript - why



- **Created at Microsoft (2012)**
 - by Anders Hejlsberg (also designed C#, Delphi, TurboPascal)
- JS didn't scale well for large scale projects, it was designed for small components in browser
 - **weak + dynamic typing** - you wouldn't know about type mismatch until it crashed in production
 - **poor tooling** - IDEs couldn't reliably provide auto-complete or safe refactoring tools
 - **lack of structure** - JS lacked native classes, modules, and interfaces which made managing millions of lines of code very difficult
- TS is **A typed superset of JavaScript** - all valid JS is valid TS



Type system



- **static + strong typing**
 - types are checked at compile time, errors caught before your code runs.
 - TS types exist only at compile time, they are stripped in JS
- **structural typing**
 - two objects are compatible if they have the same shape — not because they share a class/name.
- **type inference**
 - TS automatically figures out types when you don't specify them
- **generics**
 - reusable components that work with many types
- **gradual adoption**
 - types can be added incrementally, .js files are valid .ts files.
- **type operators**
 - very rich, allows to construct derived types and define type constraints



Type stripping



```
// TypeScript source code (.ts)
function greet(name: string): string {
    return `Hello, ${name}!`;
}
```

```
let user: string = "Alice";
let age: number = 30;
```

```
// Compiled JavaScript (.js)
function greet(name) {
    return `Hello, ${name}!`;
}
```

```
let user = "Alice";
let age = 30;
```



Structural Typing



```
type Person = {
  name: string;
  age: number;
}
interface Employee {
  name: string;
  age: number;
}

function introduce(person: Person) {
  console.log(`${person.name}, ${person.age}`);
}

const alice: Person = { name: "Alice", age: 30 };
const bob: Employee = { name: "Bob", age: 25 };
const charlie = { name: "Charlie", age: 35, department: "Engineering" }; // Extra property

introduce(alice);    // ✅ Works (exact Person)
introduce(bob);     // ✅ Works (Employee has same shape)
introduce(charlie); // ✅ Works (has name + age, extra is fine)

const david = { name: "David" }; // missing 'age'
introduce(david);    // ❌ Error: Property 'age' is missing
```



Type inference



```
let name = "Alice";           // inferred: string
let age = 30;                  // inferred: number
let isActive = true;          // inferred: boolean
let items = [1, 2, 3];         // inferred: number[]

let result = name + age;       // inferred: string (number gets coerced)

function multiply(a: number, b: number) {
  return a * b;                // inferred return type: number
}

const users = ["Alice", "Bob", "Charlie"];
users.forEach((user) => {
  console.log(user.toUpperCase()); // ✅ TypeScript knows user has .toUpperCase()
});

let mixed = [1, "hello", true]; // inferred: (string | number | boolean)[]

let something;                 // inferred: any (avoid this!)
something = "hello";
something = 42;                // No error, but 'any' disables type checking
```

```
// 1. GENERIC FUNCTION
function identity<T>(value: T): T {
    return value;
}

// 2. GENERIC INTERFACE
interface Box<T> {
    value: T;
    getValue(): T;
    setValue(value: T): void;
}

// 3. GENERIC CLASS
class Stack<T> {
    private items: T[] = [];
    push(item: T): void { this.items.push(item); }
    pop(): T | undefined { return this.items.pop(); }
}

// 4. MULTIPLE TYPE PARAMETERS
function pair<A, B>(first: A, second: B): [A, B] {
    return [first, second];
}
```

```
// 5. GENERIC CONSTRAINT
interface HasLength { length: number; }
function logLength<T extends HasLength>(item: T): T {
    console.log(item.length);
    return item;
}

// 6. GENERIC DEFAULT TYPE
interface ApiResponse<T = any> {
    data: T;
    status: number;
}

// 7. GENERIC TYPE ALIAS
type Result<T, E = Error> =
    { success: true; value: T } |
    { success: false; error: E };
```



Gradual adoption



```
// Step 1: Start with plain JS (existing code)
// user.js
function getUserData(id) {
  return fetch(`/api/users/${id}`)
    .then(response => response.json())
    .then(data => {
      return {
        id: data.id,
        name: data.name,
        email: data.email
      };
    });
}

function displayUser(user) {
  console.log(`${user.name} (${user.email})`);
}

let userId = 123;
getUserData(userId).then(displayUser);
```

```
// Step 2: Rename to .ts - it STILL works
// user.ts - same code, no changes needed!
function getUserData(id) {
  return fetch(`/api/users/${id}`)
    .then(response => response.json())
    .then(data => {
      return {
        id: data.id,
        name: data.name,
        email: data.email
      };
    });
}

function displayUser(user) {
  console.log(`${user.name} (${user.email})`);
}

let userId = 123;
getUserData(userId).then(displayUser);
// ✅ COMPILES! TypeScript didn't complain
```

```
// Step 3: Gradually add types
// (file-by-file, function-by-function)
// user.ts - with types
interface User {
  id: number;
  name: string;
  email: string;
}

// Add return type annotation
function getUserData(id: number): Promise<User> {
  return fetch(`/api/users/${id}`)
    .then(response => response.json())
    .then(data => {
      return {
        id: data.id,
        name: data.name,
        email: data.email
      };
    });
}

// Add parameter type
function displayUser(user: User): void {
  console.log(`${user.name} (${user.email})`);
}

let userId: number = 123; // Type added
getUserData(userId).then(displayUser);
```

```
// Union: Can be either
type Status = "loading" | "success" | "error";
let current: Status = "loading"; // ✓
current = "success";             // ✓
current = "failed";              // ✗ Error

// Intersection: Combines everything
type Person = { name: string };
type Employee = { id: number };
type Worker = Person & Employee; // { name: string; id: number }

// Optional: May not exist
interface Config {
  timeout?: number; // Can be omitted
}
```



Key Operators



```
interface Product {  
  id: number;  
  name: string;  
  price: number;  
}  
  
// keyof - all property names  
type ProductKeys = keyof Product; // "id" | "name" | "price"  
  
// typeof - capture runtime value's type  
const config = { port: 3000, host: "localhost" };  
type ConfigType = typeof config; // { port: number; host: string }  
  
// Indexed access - property type  
type PriceType = Product["price"]; // number  
  
// Nested indexed access  
type Nested = { a: { b: { c: string } } };  
type CType = Nested["a"]["b"]["c"]; // string
```



Conditional & Mapped Operators



```
// Conditional type (type-level if)
type Awaited<T> = T extends Promise<infer U> ? U : T;
type A = Awaited<Promise<string>>; // string
type B = Awaited<number>;          // number

// Mapped type (transform each property)
type Nullable<T> = { [P in keyof T]: T[P] | null };
interface User { id: number; name: string; }
type NullableUser = Nullable<User>; // { id: number | null; name: string | null }

// Built-in mapped types (utility types)
type ReadonlyUser = Readonly<User>; // { readonly id: number; readonly name: string }
type PartialUser = Partial<User>;   // { id?: number; name?: string }
type PickUser = Pick<User, "id">;    // { id: number }

// Infer (extract return type)
type GetReturnType<T> = T extends (...args: any[]) => infer R ? R : never;
// Type can also describe a function
function greet(): string { return "hello"; }
type Result = GetReturnType<typeof greet>; // string
```



Operators - concrete example



```
// Real-world: Type-safe event system
type EventMap = {
  click: { x: number; y: number };
  keypress: { key: string; ctrlKey: boolean };
  focus: { target: Element };
};

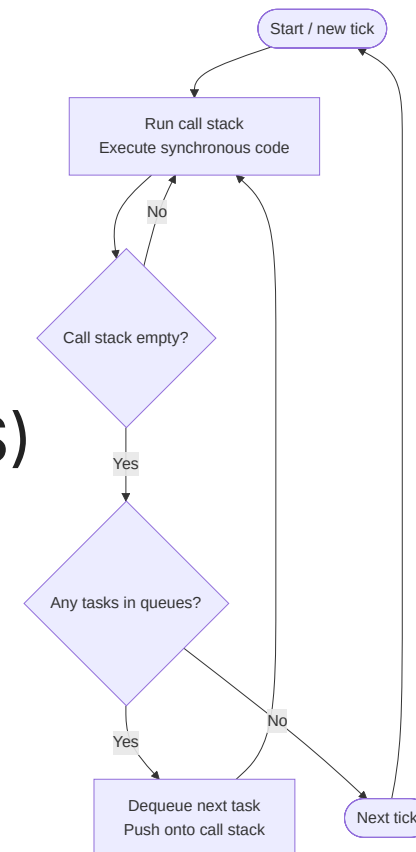
// Extract event names as union type
type EventName = keyof EventMap; // "click" | "keypress" | "focus"

// Generic event handler type using indexed access
type EventHandler<T extends EventName> = (data: EventMap[T]) => void;

// Make all event handlers optional using mapped + utility types
type EventHandlers = {
  [E in EventName]?: EventHandler<E>;
};

// Usage
const handlers: EventHandlers = {
  click: (data) => console.log(data.x, data.y),
  // keypress is optional
};
```

- JS programming model - single-threaded
 - sync code non-interruptable and non-blocking
 - no shared memory
 - no locks / deadlocks / worries :)
- event loop
 - allows I/O to be non-blocking by using callbacks (JS)
 - libuv threadpool for async OS operations
 - manages task queue of registered callbacks (JS)
 - single callback executes completely
 - CPU bound work starves event loop





Concurrency - model



Callback (Node.js original style)

```
import fs from 'fs';

fs.readFile('config.json', 'utf8', (err, data) => {
  if (err) {
    console.error('Error reading file:', err);
    return;
  }
  console.log('File content:', data);
});

console.log('This runs before file is read');

// Callback hell example
fs.readFile('a.json', 'utf8', (err, a) => {
  fs.readFile('b.json', 'utf8', (err, b) => {
    fs.readFile('c.json', 'utf8', (err, c) => {
      // pyramid of doom
    });
  });
});
```

Promise with .then() (ES2015)

```
import fs from 'fs/promises';

fs.readFile('config.json', 'utf8')
  .then((data) => {
    console.log('File content:', data);
  })
  .catch((err) => {
    console.error('Error reading file:', err);
  });

console.log('This runs before file is read');

// Chaining instead of nesting
fs.readFile('a.json', 'utf8')
  .then((a) => fs.readFile('b.json', 'utf8'))
  .then((b) => fs.readFile('c.json', 'utf8'))
  .then((c) => console.log(c))
  .catch((err) => console.error(err));
```

Async/Await (ES2017)

```
import fs from 'fs/promises';

async function readConfig() {
  try {
    const data = await fs.readFile('config.json', 'utf8');
    console.log('File content:', data);
  } catch (err) {
    console.error('Error reading file:', err);
  }
}

readConfig();
console.log('This runs before file is read');

// Sequential reads, clean and flat
async function readAll() {
  try {
    const a = await fs.readFile('a.json', 'utf8');
    const b = await fs.readFile('b.json', 'utf8');
    const c = await fs.readFile('c.json', 'utf8');
    console.log(a, b, c);
  } catch (err) {
    console.error(err);
  }
}
```

- **Worker threads** - true threads, separate v8 instances and memory heaps, message passing(SharedArrayBuffer + Atomics), offloading CPU-intensive work (image processing, encryption)
- **Web workers** - browser equivalent, no access to DOM, to keep UI responsive
- **Cluster mode** - multiple OS processes, OS distributes incoming network connections
- **Child process** - more general spawning of OS process - nodejs/python/shell

	Worker threads	Web Workers	Cluster	Child process
Environment	Node.js	Browser	Node.js	Node.js
Unit	Thread	Thread	Process	Process
Shared memory	opt-in	opt-in	✗	✗
Use case	CPU-bound tasks	Keep UI responsive	Scale HTTP servers	External tools
Crash isolation	✗	✗	✓	✓

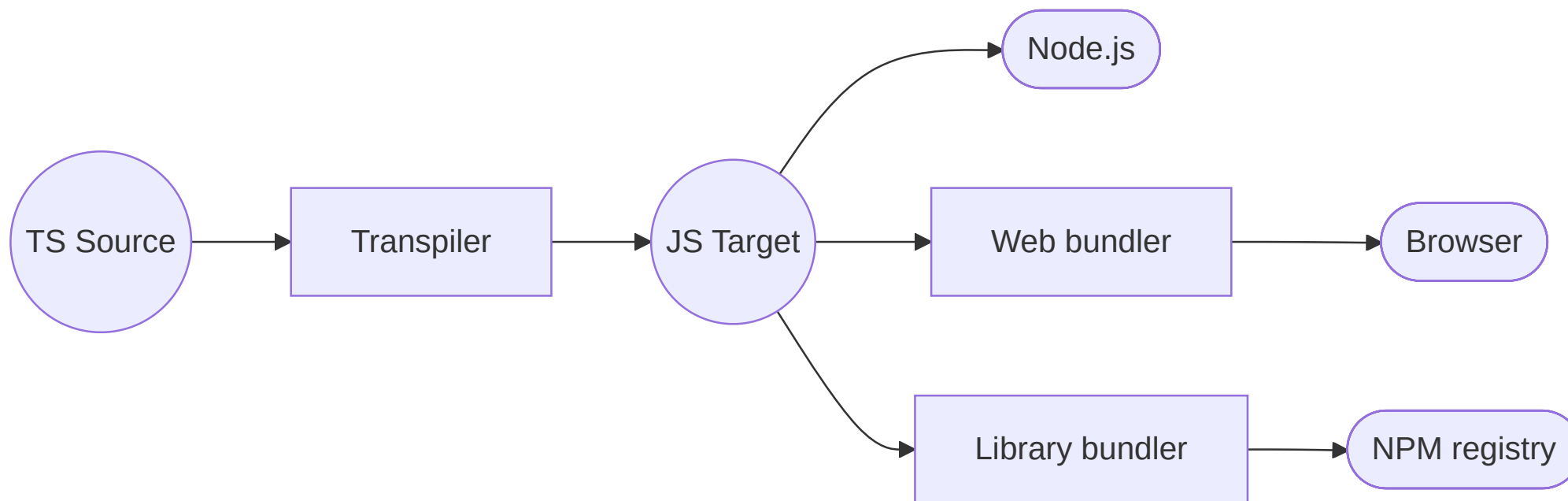
- build artifact is portable JS code
- requires JS **engine**
 - executes JS
 - parses JS, compiles to machine code (JIT), memory + GC
- lives inside **runtime**
 - connects engine to real world
 - environments specific APIs, readFile(), fetch(), DOM
 - provides event loop



Platforms



Runtime	Engine	Environments
Node.js	V8	Server, CLI, desktop
Deno	V8	Server, CLI, edge
Bun	JavaScriptCore (Nitro)	Server, CLI
Browser(s)	V8 / SpiderMonkey / JavaScriptCore	Client
Cloudflare workers	V8	Edge / CDN nodes
Vercel Edge	V8	Edge / CDN nodes
Electron	V8 + Chromium	Desktop apps
React Native	Hermes / JSCore	iOS, Android
Espruino	Custom	Microcontrollers
Kaluma	JerryScript	Microcontrollers





Transpiler - tsc



- Parameters describing runtime
 - **target** - JS version: **ES3, ES5, ... ES2023, ESNext**
 - **module** - the module system: **CommonJS, ES6, ..., Node16**
 - **lib** - APIs included in environment: **ES5, DOM, ... ,WebWorker**
- Transpile process
 - type checking (*: string, <T>, type X =*)
 - type information removed and not available in runtime
 - transforms TS constructs into JS constructs (*enum, @decorator*)
 - transforms JS constructs between language versions (*await, import, ??*)
- slowest one because of type checking



Web bundler



- Additional step after TS transpiler
- Optimizes JS for delivery to browser
 - bundles code and referenced libraries into single file
 - minifies code - shortens non-public names, removes blank spaces and new lines
 - tree shaking - removes unused code
- Also many options: **webpack**, vite, parcel, rspack, ...



Library bundler



- Packages in NPM are all JS
- Declaration files **.d.ts** used by TS consumers
 - Alternative is **@types/<package>** registry (*DefinitelyTyped repository*)
- Multiple module format outputs **cjs/mjs**
- Tools: rollup, tsup

- Lots of options, overlap of responsibilities and dependencies
- **swc** (Rust)
 - **transpiler**, very fast, strips types, no type validation
- **babel**
 - general purpose **transpiler** based on plugins, has @babel/preset-typescript
- **esbuild** (Go)
 - **transpiler**, very fast, strips types, no type validation
 - **bundler**, also handles CSS, images, etc...
- **tsup**
 - internally uses **esbuild** transpiler
 - **bundler**, mostly used for libraries
- **rollup**
 - transpiler through plugins, most common **tsc** or **swc** based
 - **bundler**, also used for libraries

- **webpack**
 - mature, very flexible and complex
 - transpilation via loaders, can be **tsc**, **babel**, etc.
 - **bundler**, configurable via modules
 - supports dev server, file watchers, HMR
- **vite**
 - build tool and dev server with HMR
 - **esbuild** for transpiling
 - **rollup** for bundling
 - often used in popular frontend frameworks: **react**, vue, svelte
- **parcel**
 - zero config bundler
 - internally uses **swc** for fast transpilation
 - out of the box HMR,

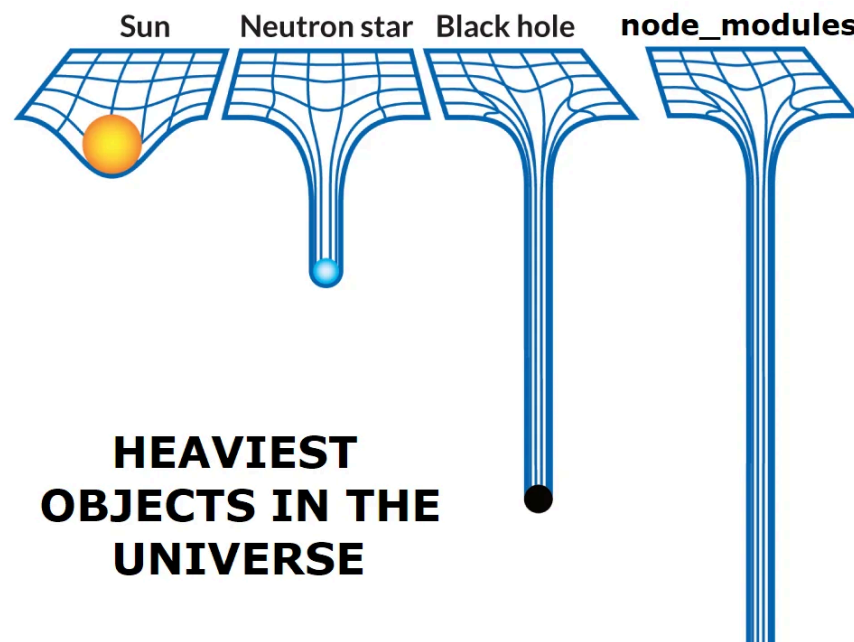
- **ECMAScript** built-ins
 - Array, Map, Set, Promise, Date, Math
 - JSON.parse(), JSON.stringify()
 - fetch()
- **Runtime-specific** standard libraries

Capability	Node.js	Deno	Browser
File system	fs	Deno.readFile	✗
HTTP server	http	Deno.serve	✗
HTTP client	fetch	fetch	fetch
Environment vars	process.env	process.env	✗
DOM / UI	✗	✗	document, window
Graphics	✗	✗	Canvas, WebGL, WebGPU

- **package.json**
 - manifest file
- **semver**, semantic versioning
 - ^4.18.0 ← minor+patch 4.x.x
 - ~4.18.0 ← patch 4.18.x
 - 4.18.0 ← exact version
- **npm install**
- **package-lock.json**
 - snapshot of resolved versions
- **node_modules**
 - recursive storage of packages

```
{
  "name": "my-library",
  "version": "1.0.1",
  "dependencies": {
    "express": "^4.18.0"
  },
  "devDependencies": {
    "typescript": "^5.0.0"
  },
  "scripts": {
    "build": "tsc",
    "start": "node dist/index.js"
  }
}
```

- local storage of installed modules
- famously enormous because of transitive dependencies





Package management - tools

- **npm install**
 - original, ships with Node.js
 - slow historically
- **yarn install**
 - 2016 Facebook
 - fix npm's speed
- **pnpm**
 - packages reused locally across projects
- **bun**
 - runtime with built-in package manager, fastest (Zig → Rust powered by Claude)
- all of them use package registry backend
 - default **registry.npmjs.org**
 - alternatives: GitHub packages, AWS CodeArtifact, ...

- competitive on I/O, slower on CPU-bound and latency-sensitive workloads
- where it's fast
 - I/O bound workloads - event loop handles well
 - JSON processing - native parser in C++
 - long-running server - JIT warmup
- where it struggles
 - CPU-bound work - single threaded, event loop blocked
 - consistent latency - GC pauses and JIT deoptimization
 - memory efficiency - V8 heap overhead
 - startup time - parsing + initial JIT compilation



Go & Typescript

```
$ go version  
go version go1.26.3 linux/arm64
```

```
$ node -v  
v26.3.0
```

```
$ tsc -v  
Version 6.0.3
```



Demo: Hello & run speed



```
package main

import "fmt"

func main() {
    fmt.Println("Hello from Go")
}
```

```
$ time go run main.go
Hello from Go
real    0m0.103s
user    0m0.000s
sys     0m0.000s
```

TS

```
const msg: string = "Hello from TypeScript"
console.log(msg);
```

```
$ time node main.ts
Hello from TypeScript
real    0m0.112s
user    0m0.000s
sys     0m0.000s
```



Demo: Hello & run speed



```
package main

import "fmt"

func main() {
    fmt.Println("Hello from Go")
}
```

```
$ go build -o hello
time ./hello
Hello from Go
real    0m0.010s
user    0m0.000s
sys     0m0.000s
```

TS

```
const msg: string = "Hello from TypeScript"
console.log(msg);
```

```
$ bun build --compile main.ts --outfile hello
time ./hello
    [3ms] bundle 1 modules
  [195ms] compile hello
Hello from TypeScript
real    0m0.206s
user    0m0.000s
sys     0m0.000s
```



Go, TS/JS to a binary?

Runtime	Build command	Build time	Exec (avg/10)	Binary size
Go	<code>go build</code>	150 ms	1.8 ms	2.3 MB
Go (run)	<code>go run</code> (compile+run)	-	158 ms	no artifact
Bun	<code>bun build --compile</code>	272 ms	15 ms	91 MB
Deno	<code>deno compile</code>	573 ms	30 ms	90 MB
Node SEA	<code>--experimental-sea</code> + postject	~4830 ms	23 ms	124 MB
Node	<code>node app.ts</code> (no build)	-	60 ms	needs runtime

Same hello program, measured here (avg of 10 runs). go run recompiles each time (like node); the built binary runs in ~2 ms.

** Hello world, might not be the best example*



Demo: Type safety



```
package main

import "fmt"

func main() {
    // type error
    var n int = "not a number"
    fmt.Println(n)
}
```

```
$ time go build main.go
# command-line-arguments
./main.go:7:15: cannot use "not a number" (untyped string constant) as int value in variable declaration
real    0m0.054s
user    0m0.000s
sys     0m0.000s
[exit 1]
```

TS

```
// type error
let n: number = "not a number";
console.log(n);
```

```
$ time tsc --noEmit main.ts
main.ts(2,5): error TS2322: Type 'string' is not assignable to type 'number'.
real    0m1.813s
user    0m0.000s
sys     0m0.000s
[exit 2]
```



Demo: Concurrency



```
"sync"
)

func main() {
    var wg sync.WaitGroup
    for i := 1; i <= 3; i++ {
        wg.Add(1)
        go func(n int) {
            defer wg.Done()
            fmt.Println("worker", n, "done")
        }(i)
    }
    wg.Wait()
    fmt.Println("all done")
}
```

```
$ go run main.go
worker 3 done
worker 1 done
worker 2 done
all done
```

TS

```
const work = (n: number): Promise<string> =>
    new Promise(res =>
        setTimeout(() => {
            console.log(`worker ${n} done`);
            res();
        })
    )

const main = async () => {
    await Promise.all([1, 2, 3].map(work));
    console.log("all done");
};

main();
```

```
$ node main.ts
worker 1 done
worker 2 done
worker 3 done
all done
```



Demo: JSON handling



```
type User struct {
    Name string `json:"name"`
    Age  int    `json:"age"`
}

func main() {
    in := []byte(`{"name":"Ada","age":36}`)
    var u User
    json.Unmarshal(in, &u)
    out, _ := json.Marshal(u)
    fmt.Println(u.Name, u.Age)
    fmt.Println(string(out))
}
```

```
$ go run main.go
Ada 36
{"name":"Ada","age":36}
```

TS

```
interface User {
    name: string;
    age: number;
}

const input = '{"name":"Ada","age":36}';
const u: User = JSON.parse(input);
console.log(u.name, u.age);
console.log(JSON.stringify(u));
```

```
$ node main.ts
Ada 36
{"name":"Ada","age":36}
```



Demo: Error handling



```
import (  
    "fmt"  
    "strconv"  
)  
  
func main() {  
    n, err := strconv.Atoi("not-a-number")  
    if err != nil {  
        fmt.Println("handled error:", err)  
        return  
    }  
    fmt.Println("parsed:", n)  
}
```

```
$ go run main.go  
handled error: strconv.Atoi: parsing "not-a-number": invalid syntax
```

TS

```
function parse(s: string): number {  
    const n = Number(s);  
    if (Number.isNaN(n)) {  
        throw new Error(`cannot parse: ${s}`);  
    }  
    return n;  
}  
  
try {  
    console.log("parsed:", parse("not-a-number"))  
} catch (e) {  
    console.log("handled error:", (e as Error).message)  
}
```

```
$ node main.ts  
handled error: cannot parse: not-a-number
```



Demo: Null safety



```
type User struct{ Name string }

func find(id int) *User {
    if id == 1 {
        return &User{Name: "Ada"}
    }
    return nil // nothing found
}

func main() {
    u := find(2)
    if u == nil {
        fmt.Println("no user (nil check)")
        return
    }
    fmt.Println(u.Name)
}
```

```
$ go run main.go
no user (nil check)
```

TS

```
interface User { name: string; }

function find(id: number): User | undefined {
    return id === 1 ? { name: "Ada" } : undefined;
}

const u = find(2);
// optional chaining + nullish coalescing
console.log(u?.name ?? "no user (undefined check)");
```

```
$ node main.ts
no user (undefined check)
```



HTTP server



```
package main

import (
    "fmt"
    "net/http"
)

func handler(w http.ResponseWriter, r http.Request) {
    fmt.Fprintln(w, "Hello from Go (net/http)")
}

func main() {
    http.HandleFunc("/", handler)
    http.ListenAndServe("localhost:8099", nil)
}
```



```
import { createServer } from "http";

const srv = createServer((_req, res) => {
    res.end("Hello from Node (http)\n");
});

srv.listen(8098, "localhost");
```

Thank you!

Questions & Answers

